GIÁO ÁN HỌC THUẬT & KỊCH BẢN THUYẾT TRÌNH CHI TIẾT (SLIDE 1 ➔ 7)

QUẢN LÝ TRẠNG THÁI VỚI PROVIDER VÀ CHANGENOTIFIER TRONG FLUTTER

Ứng dụng thực tế Course Registration & Phòng thí nghiệm Provider Lab

Giảng viên hướng dẫn: TS. Nguyễn Duy Nhật Viễn

Sinh viên báo cáo:

- **Đặng Công Thắng** – Lớp 23KTMT2 – MSSV: 106230061 (Phụ trách thuyết trình Slide 1–7)
- **Võ Văn Thái** – Lớp 23KTMT1 – MSSV: 106230027 (Phụ trách thuyết trình Slide 8–14)

Mã nguồn GitHub: `https://github.com/cityengin/course-registration.git`

MỤC TIÊU NỬA ĐẦU BÀI BÁO CÁO (SLIDE 1 ĐẾN 7):

Giảng giải từ nền tảng cơ bản nhất của Flutter (State là gì, hạn chế của setState, Prop Drilling, Over-rebuilding) đến mô hình kiến trúc phân lớp chuẩn mực, luồng dữ liệu một chiều, thiết kế RegistrationNotifier chứa toàn bộ nghiệp vụ, phân biệt Provider Toàn cục vs Cục bộ, và phân tích sâu 4 cách đọc State trong Provider.

Slide	Chủ đề bài giảng	Mục tiêu truyền tải	Thời lượng
Slide 1	Trang tiêu đề & Giới thiệu chuyên đề	Lời chào, đặt vấn đề State Management cuốn hút	~45 giây
Slide 2	Phân công nhiệm vụ	Tính bổ trợ: Nghiệp vụ thực tế (Thắng) vs Thực nghiệm Lab (Thái)	~45 giây
Slide 3	Tổng quan đề tài & Tại sao cần Provider?	Vấn đề của setState (Prop Drilling & Over-rebuild), 3 khối giải pháp	~1m15s
Slide 4	Kiến trúc phân lớp & Luồng 1 chiều	Tách biệt 3 tầng UI - State - Data; Luồng dữ liệu khép kín	~1m15s
Slide 5	RegistrationNotifier & Logic nghiệp vụ	Encapsulation (unmodifiable), fold(), Conditional Notification	~1m30s
Slide 6	Provider Toàn cục vs Cục bộ	Global (main.dart) vs Scoped (tự dispose giải phóng RAM)	~1m15s
Slide 7	Bốn cách đọc State trong Provider	Trục thu hẹp: watch, Consumer, select, read & Quy tắc vàng	~1m45s

Slide	Chủ đề bài giảng	Mục tiêu truyền tải	Thời lượng
TỔNG THỜI LƯỢNG NỬA ĐẦU BÁO CÁO (SLIDE 1 - 7)			**~ 8 phút 30 giây**

SLIDE 1: MỞ ĐẦU & GIỚI THIỆU CHUYÊN ĐỀ (Thời lượng: ~45 giây)

Trang tiêu đề chính thức: Giới thiệu đề tài, giảng viên hướng dẫn, nhóm sinh viên, đơn vị ĐHBK Đà Nẵng, link mã nguồn.

🎙️ LỜI THOẠI THUYẾT TRÌNH (SPEAKER NOTES):

[CÚI ĐẦU CHÀO, PHONG THÁI TỰ TIN, MẮT NHÌN BAO QUÁT GIẢNG VIÊN VÀ CÁC BẠN]

"Kính thưa thầy Nguyễn Duy Nhật Viễn cùng toàn thể các bạn sinh viên đang có mặt trong buổi báo cáo ngày hôm nay!

Em tên là **Đặng Công Thắng**, đại diện nhóm gồm em và bạn **Võ Văn Thái**, xin phép được trình bày chuyên đề nghiên cứu môn Lập trình đa nền tảng với đề tài: **'Quản lý trạng thái với Provider và ChangeNotifier trong Flutter'**, thông qua sản phẩm thực tế là ứng dụng **Đăng ký học phần – Course Registration** và phòng thí nghiệm đo lường hiệu năng **Provider Lab**.

Khi mới học Flutter, chúng ta thường làm quen với việc dựng giao diện qua các Widget. Nhưng khi ứng dụng lớn dần lên, có nhiều màn hình và luồng dữ liệu phức tạp, câu hỏi lớn nhất là: *Làm thế nào để các Widget ở các vị trí khác nhau chia sẻ dữ liệu đồng bộ, dễ bảo trì và không làm giảm hiệu năng ứng dụng?* Đó chính là bài toán **Quản lý trạng thái (State Management)** mà nhóm em sẽ làm sáng tỏ ngay sau đây.

Toàn bộ mã nguồn đã được nhóm công khai trên GitHub tại địa chỉ `github.com/cityengin/course-registration`."

SLIDE 2: PHÂN CÔNG NHIỆM VỤ & TINH THẦN CỘNG TÁC (Thời lượng: ~45 giây)

Bố cục 2 mảng: Đặng Công Thắng (52% - Nghiệp vụ & Kiến trúc) | Võ Văn Thái (48% - Thực nghiệm Lab đo rebuild).

🎙️ LỜI THOẠI THUYẾT TRÌNH (SPEAKER NOTES):

[CHUYỂN SLIDE, CHỈ TAY HƯỚNG VỀ HAI CỘT PHÂN CÔNG]

"Ở slide thứ 2, em xin tóm lược bảng phân công công việc của nhóm:

Để đề tài vừa có một ứng dụng nghiệp vụ hoàn chỉnh, vừa có công cụ thực nghiệm đo lường khoa học, nhóm em đã phân chia hai mảng nhiệm vụ tương hỗ rất rõ ràng:

- Về phần em – **Đặng Công Thắng**, em chịu trách nhiệm thiết kế kiến trúc phân lớp, mô hình dữ liệu, xây dựng Notifier trung tâm quản lý nghiệp vụ đăng ký tín chỉ (từ 14 đến 24 tín chỉ), hoàn thiện 2 màn hình chính và các kịch bản demo kiểm thử tự động.

- Về phần bạn **Võ Văn Thái**, bạn Thái tập trung phát triển phòng thí nghiệm **Provider Lab** độc lập nhằm đo đếm chính xác số lần hàm `build()` thực thi, từ đó so sánh trực quan giữa `setState` truyền thống với 4 cách đọc dữ liệu khác nhau của Provider.

Hai phần việc tương hỗ chặt chẽ với tỷ lệ đóng góp là 52% và 48%."

SLIDE 3: TỔNG QUAN ĐỀ TÀI & TẠI SAO CẦN PROVIDER? (Thời lượng: ~1 phút 15 giây)

Vấn đề: Cùng 1 dữ liệu hiển thị tại 4 nơi. setState gây Prop Drilling & Over-rebuild. Giải pháp 3 khối thống nhất.

📖 KIẾN THỨC NỀN TẢNG TỪ CƠ BẢN NHẤT (PEDAGOGICAL FOUNDATION):

State (Trạng thái) là gì? Là thông tin có thể thay đổi trong suốt vòng đời của app và quyết định giao diện tại một thời điểm (VD: danh sách môn đã chọn, số tín chỉ). Khi State đổi, giao diện phải phản hồi tương ứng.

Hạn chế chí mạng của `setState()` trong dự án thực tế:

- **Prop Drilling (Chuyền tham số qua nhiều tầng):** `setState()` chỉ có tác dụng cục bộ trong 1 `StatefulWidget`. Muốn đưa dữ liệu lên AppBar hoặc sang màn hình khác, ta phải truyền biến và callback qua hàng loạt Widget trung gian không liên quan.
- **Over-rebuilding (Vẽ lại cả cây Widget):** Mỗi lần gọi `setState()` ở Widget cha, toàn bộ cây con cháu bên dưới bị ép chạy lại hàm `build()`, gây lãng phí tài nguyên CPU/GPU và tụt FPS.

🎙️ LỜI THOẠI THUYẾT TRÌNH (SPEAKER NOTES):

"Thưa thầy và các bạn, để hiểu rõ lý do nhóm lựa chọn Provider, chúng ta hãy xuất phát từ **bản chất cơ bản nhất của cây giao diện Flutter**:

Hãy hình dung: Khi sinh viên nhấn 'Đăng ký' môn Lập trình đa nền tảng, ngay lập tức có 4 nơi cần cập nhật đồng thời:

1. Huy hiệu tín chỉ trên thanh AppBar góc trên phải nhảy số.
2. Thẻ môn học đó đổi màu và chuyển nút thành 'Hủy môn'.
3. Thanh tiến độ dưới chân màn hình tăng lên.
4. Ở màn hình 'Các môn đã đăng ký', môn này cũng phải lập tức xuất hiện.

Nếu dùng cách mặc định sơ khai là `setState()`: Widget cha trên đỉnh phải giữ biến State, sau đó chuyền biến và callback qua 4–5 tầng widget trung gian. Các widget ở giữa hoàn toàn không cần dữ liệu đó nhưng vẫn phải 'cõng' tham số giùm. Hơn thế nữa, mỗi lần gọi `setState()`, cả cây con bị ép vẽ lại từ đầu, gây lãng phí CPU và giật lag trên thiết bị yếu.

Vì vậy, nhóm em xây dựng giải pháp Provider gồm 3 khối module: Ứng dụng thực tế chuẩn đào tạo ĐHBK, Bảng Demo Panel quan sát nhật ký notify, và Phòng thí nghiệm Provider Lab đo đếm thực tế – bảo chứng bởi 7 bài test tự động đạt 100%."

💡 **Câu hỏi phản biện có thể gặp:** "*Tại sao nhóm chọn Provider mà không dùng GetX hay Bloc?*"
Gợi ý trả lời: "*Thưa thầy, Provider là giải pháp chính thức được Google khuyến nghị trong tài liệu Flutter. Nó xây dựng trực tiếp trên nền tảng InheritedWidget của Flutter, có tính cấu trúc cao, không phá vỡ cây Widget như GetX và cú pháp gọn gàng hơn rất nhiều so với Bloc.*"

SLIDE 4: KIẾN TRÚC PHÂN LỚP & LUỒNG DỮ LIỆU MỘT CHIỀU (Thời lượng: ~1 phút 15 giây)

Mô hình 3 tầng: UI Layer, State Layer, Data Layer. Luồng dữ liệu một chiều Unidirectional Data Flow.

📖 KIẾN THỨC NỀN TẢNG TỪ CƠ BẢN NHẤT:

Separation of Concerns (Tách biệt trách nhiệm): Nếu viết logic tính toán tín chỉ ngay trong nút bấm của Widget, mã nguồn sẽ trở thành 'Spaghetti code', không thể viết Unit Test độc lập. Do đó, nhóm tách biệt hoàn toàn:

- **UI:** Chỉ hiển thị giao diện và bắt sự kiện chạm (onPressed).
- **State:** Lưu trữ dữ liệu sống và chứa toàn bộ logic kiểm tra điều kiện.
- **Data:** Lưu cấu trúc dữ liệu bất biến (Model Course).

🎙️ LỜI THOẠI THUYẾT TRÌNH (SPEAKER NOTES):

[CHỈ TAY VÀO SƠ ĐỒ LUỒNG DỮ LIỆU TRÊN MÀN HÌNH]

"Ở slide 4, em xin kính mời thầy và các bạn cùng quan sát linh hồn kiến trúc của ứng dụng: **Mô hình phân lớp và Luồng dữ liệu một chiều**.

Trong kỹ thuật phần mềm chuẩn mực, nguyên tắc hàng đầu là tách biệt trách nhiệm:

- Bên tay trái là **Lớp Giao diện (UI)**: Thẻ môn học hay nút bấm chỉ làm đúng một việc: lắng nghe thao tác của người dùng. Khi người dùng bấm 'Đăng ký', Widget không tự ý tính toán, mà chỉ dùng lệnh `context.read` gửi thông điệp yêu cầu Notifier xử lý.

- Ở trung tâm là **RegistrationNotifier**: Đây là 'bộ não' của ứng dụng. Notifier kiểm tra điều kiện nghiệp vụ. Nếu hợp lệ, Notifier cập nhật dữ liệu và bắn ra sự kiện: `notifyListeners()`.

- Ngay lập tức, **ChangeNotifierProvider** bọc ở gốc ứng dụng sẽ đánh thức các Widget con đăng ký lắng nghe nó ở bên phải: Huy hiệu `CreditBadge` dùng `watch` cập nhật số tín chỉ; Thẻ `CourseTile` dùng `select` đổi nút; và `SummaryBar` dùng `Consumer` vẽ lại thanh tiến độ.

Dữ liệu đi theo đúng một chiều khép kín: **UI phát sinh hành động $\rightarrow$ State biến đổi $\rightarrow$ UI cập nhật**. Không có thay đổi ngầm từ bên ngoài, giúp hệ thống hoạt động ổn định và cực kỳ dễ gỡ lỗi."

SLIDE 5: REGISTRATIONNOTIFIER – TRÁI TIM LOGIC NGHIỆP VỤ (Thời lượng: ~1 phút 30 giây)

4 nguyên tắc: Đóng gói (private & unmodifiable), Trạng thái dẫn xuất (fold), Ràng buộc 14–24 TC, Notify có điều kiện.

📖 KIẾN THỨC NỀN TẢNG TỪ CƠ BẢN NHẤT:

ChangeNotifier hoạt động như thế nào? Kế thừa từ `flutter/foundation.dart`, nó cài đặt mẫu thiết kế Observer. Khi gọi `notifyListeners()`, nó duyệt qua danh sách các listener đã đăng ký để báo hiệu cần vẽ lại. Do đó, nếu không có thay đổi thật sự, tuyệt đối không được gọi hàm này.

🎙️ LỜI THOẠI THUYẾT TRÌNH (SPEAKER NOTES):

"Đi sâu vào chi tiết cài đặt của bộ não ứng dụng ở Slide 5: Lớp `RegistrationNotifier` kế thừa từ `ChangeNotifier`.

Ở đây, nhóm em tuân thủ nghiêm ngặt 4 nguyên tắc kỹ thuật chuẩn mực:

1. **Thứ nhất là Tính đóng gói:** Danh sách `_registered` là private với dấu gạch dưới. Bên ngoài chỉ có thể đọc qua getter `List.unmodifiable`. Điều này đảm bảo tầng UI không bao giờ có thể tự ý can thiệp hay sửa trộm dữ liệu danh sách.

2. **Thứ hai là Trạng thái dẫn xuất (Derived State):** Chúng ta không tạo biến độc lập `int totalCredits = 0;` rồi cộng dồn thủ công. Ở đây, `totalCredits` là getter tính toán tức thì bằng hàm `fold()` từ danh sách. Danh sách học phần là nguồn chân lý duy nhất (Single Source of Truth), loại bỏ hoàn toàn nguy cơ lệch dữ liệu.

3. **Thứ ba là Ràng buộc quy chế đào tạo:** Toàn bộ quy định 14–24 tín chỉ và kiểm tra chống trùng môn đều được gom tập trung tại đây.

4. **Và thứ tư – nguyên tắc tối ưu then chốt: Notify có điều kiện:**
Nếu người dùng bấm đăng ký trùng môn hoặc vượt quá 24 tín chỉ, hàm lập tức trả về chuỗi báo lỗi và thoát ra. **Tuyệt đối không gọi `notifyListeners()`!** Nhờ vậy, không có bất kỳ Widget nào trên màn hình bị kích hoạt rebuild thừa thãi. Đây chính là cách nhóm tối ưu hóa hiệu năng ngay từ tầng logic nghiệp vụ."

SLIDE 6: CUNG CẤP STATE – TOÀN CỤC VS CỤC BỘ (Thời lượng: ~1 phút 15 giây)

So sánh vị trí khai báo, phạm vi truy cập, vòng đời tài nguyên và mục đích kiến trúc của Provider Toàn cục vs Cục bộ.

📖 KIẾN THỨC NỀN TẢNG TỪ CƠ BẢN NHẤT:

ChangeNotifierProvider làm gì bên dưới? Nó nhận hàm `create` để khởi tạo Notifier, quản lý vòng đời và cấp phát Notifier xuống cây Widget qua `InheritedWidget`. Khi Widget chứa Provider bị hủy (unmount), Provider tự động gọi phương thức `dispose()` của Notifier để giải phóng bộ nhớ RAM.

🎙️ LỜI THOẠI THUYẾT TRÌNH (SPEAKER NOTES):

"Sau khi đã xây dựng xong Notifier, câu hỏi tiếp theo là: **Làm thế nào để các Widget trên cây giao diện có thể tìm thấy và lấy dữ liệu từ Notifier này?** Câu trả lời nằm ở `ChangeNotifierProvider` tại Slide 6.

Trong đề tài này, nhóm em vận dụng linh hoạt cả hai mô hình cung cấp State:

1. **Thứ nhất là Provider Toàn cục (Global):** Được bọc ngoài cùng quanh `MaterialApp` tại `main.dart`. Vì sao? Vì dữ liệu đăng ký học phần cần dùng chung giữa hai màn hình khác nhau: Màn hình danh sách môn học và Màn hình học phần đã chọn. Nhờ bọc toàn cục, cả hai màn hình đều cùng truy xuất vào một thể hiện `RegistrationNotifier` duy nhất mà không cần truyền callback qua Navigator.

2. **Thứ hai là Provider Cục bộ (Scoped Provider):** Được khai báo riêng trong `provider_lab_screen.dart`. Màn hình Lab phục vụ đo đạc thực nghiệm, không liên quan đến dữ liệu đăng ký thật. Khi người dùng mở màn Lab, `LabNotifier` mới được sinh ra. Và quan trọng nhất: khi người dùng bấm Back rời khỏi màn Lab, `ChangeNotifierProvider` sẽ tự động phát hiện và gọi ngay hàm `dispose()` để thu hồi bộ nhớ RAM.

Việc kết hợp Toàn cục cho dữ liệu dùng chung và Cục bộ cho màn hình độc lập là chìa khóa giúp ứng dụng hoạt động nhẹ nhàng và chống rò rỉ bộ nhớ (memory leak)."

SLIDE 7: BỐN CÁCH ĐỌC STATE TRONG PROVIDER (Thời lượng: ~1 phút 45 giây)

Trục thu hẹp Rebuild: setState $\rightarrow$ watch $\rightarrow$ Consumer $\rightarrow$ select $\rightarrow$ read. Quy tắc vàng đặt trong build() hay callback.

🎙️ LỜI THOẠI THUYẾT TRÌNH (SPEAKER NOTES):

[NHẤN MẠNH GIỌNG, TRUYỀN CẢM HỨNG, BƯỚC CHUYỂN GIAO SANG NGƯỜI THUYẾT TRÌNH TIẾP THEO]

"Và đây chính là Slide 7 – **Slide then chốt nhất trong toàn bộ phần lý thuyết nền tảng**: Bốn phương thức đọc State trong Provider.

Nhiều bạn thắc mắc tại sao mình dùng Provider rồi mà ứng dụng vẫn bị vẽ lại quá nhiều. Câu trả lời nằm ở việc lựa chọn đúng phương thức đọc State:

1. **Đầu tiên là `context.watch<T>()`:** Đây là cách đọc toàn diện nhất. Cứ mỗi khi Notifier gọi `notifyListeners()`, toàn bộ Widget gọi hàm `watch` này sẽ bị build lại. Ta dùng nó cho những Widget nhỏ hiển thị trạng thái tổng quan, ví dụ như huy hiệu `CreditBadge` trên AppBar.

2. **Thứ hai là `Consumer<T>`:** Khi một Widget có kích thước lớn, chứa nhiều thành phần tĩnh như banner, tiêu đề, ta không muốn toàn bộ Widget đó bị vẽ lại. `Consumer` giúp ta khoanh vùng: Chỉ có hàm `builder` của nó chạy lại, còn phần giao diện tĩnh được đưa vào tham số `child` sẽ được tái sử dụng nguyên vẹn mà không tốn tài nguyên render.

3. **Thứ ba – Đỉnh cao của việc tối ưu – là `context.select<T, R>()`:** Giả sử danh sách có 10 môn học. Khi sinh viên bấm đăng ký Môn số 1, ta chỉ muốn thẻ Môn số 1 đổi trạng thái. Chín môn còn lại không hề đổi thì tuyệt đối không được phép rebuild! `context.select` giải quyết việc này xuất sắc: Nó chỉ theo dõi đúng giá trị `isRegistered(id)`. Khi tổng tín chỉ tăng nhưng trạng thái môn này không đổi, thẻ môn này hoàn toàn không rebuild.

4. **Và cuối cùng là `context.read<T>()`:** Phương thức này hoàn toàn **KHÔNG** đăng ký lắng nghe. Nó chỉ lấy tham chiếu của Notifier để gọi hàm, ví dụ `notifier.register(course)`. Do đó, nó sinh ra là để đặt bên trong các sự kiện bấm nút `onPressed`.

👉 Nhóm em xin đúc kết thành **Quy tắc vàng:** `watch`, `select` và `Consumer` dùng trong hàm `build()`; còn `context.read` luôn luôn đặt trong callback sự kiện!

Và để chứng minh cho thầy và các bạn thấy sự chênh lệch rõ rệt về số lần rebuild giữa các cách đọc này trên thực tế, sau đây em xin phép nhường lại phần trình bày tiếp theo cho bạn **Võ Văn Thái** để đưa chúng ta bước vào các Slide thực nghiệm chuyên sâu và tham quan phòng thí nghiệm Provider Lab! Xin trân trọng cảm ơn thầy và các bạn!"